Fall 2026

CS-UY 2413 (Sections A and B): Design and Analysis of Algorithms

Prof. Lisa Hellerstein
lisa.hellerstein@nyu.edu

Lecture 2

- Reading:
  - Read Chap. 2
  - Read Chap. 3 to refresh your memory

Today's topics

- Detailed analysis of Bubble Sort [book does Insertion Sort]
- Asymptotic Notation, big-Oh (O), big-Omega (Ω), theta (θ), little-o (o), little-omega (ω)
  - Formal definitions
  - Review of properties
  - Hints on how to use

Running time of Bubble Sort

But first, a story



Carl Friedrich Gauss

$$\frac{(n+1)*n}{2}$$

- Story: When Carl Friedrich Gauss was about 6 years old

$$\begin{array}{rcccccc} & 1 & + 2 & + 3 & + \cdots + & 99 & + 100 \\ + & 100 & + 99 & + 98 & \cdots + & 2 & + 1 \\ \hline & & & 101 * 100 & & & \end{array}$$

$$\Rightarrow 1+2+\cdots+100 = \frac{101*100}{2} = 101*50 = 5050$$

- Never forget, the sum of the numbers from 1 to n is

$$n(n + 1)/2$$

- But if you forget, remember this story and work it out for yourself again

- Can also write the rule using summation notation:

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$$

Now, back to algorithms analysis

Running time of Bubble Sort

Bubble Sort

```
BUBBLESORT(A, n)
1  for i = 1 to n − 1
2      for j = n downto i + 1
3          if A[j] < A[j − 1]
4              exchange A[j] with A[j − 1]
```

A is array of length n, stores the input sequence

Want to determine the (worst-case) number of basic operations executed by A

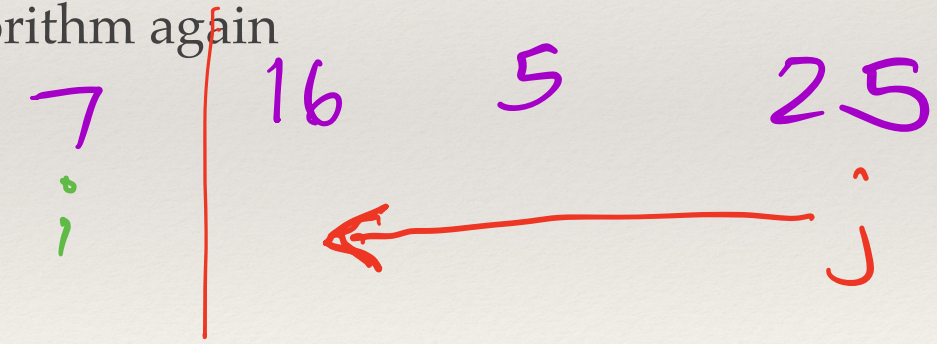
We'll express runtime as function of input size n, using big-Oh notation

Bubble Sort

```
BUBBLESORT(A, n)
1  for i = 1 to n − 1
2      for j = n downto i + 1
3          if A[j] < A[j − 1]
4              exchange A[j] with A[j − 1]
```

First, let's get feeling for algorithm again

[7, 16, 5, 25]



Bubble Sort

```
BUBBLESORT(A, n)
1  for i = 1 to n − 1
2      for j = n downto i + 1
3          if A[j] < A[j − 1]
4              exchange A[j] with A[j − 1]
```

Change of variables to make inner loop a little easier to analyze

```
BUBBLESORT(A, n)
for i = 1 to n − 1
    for k = 1 to n − i                    // k = n − j + 1 (for k=1, which are compared?)
        if A[n − k + 1] < A[n − k]
            exchange A[n − k + 1] with A[n − k]
```

Bubble Sort

BUBBLESORT(A, n)
for $i = 1$ to $n - 1$
    for $k = 1$ to $n - i$
        if $A[n - k + 1] < A[n - k]$
            exchange $A[n - k + 1]$ with $A[n - k]$

Let's say first line uses c_1 basic operations, second c_2, third c_3 and fourth c_4

Number of basic operations executed is at most: (when if condition always True)

$$c_1 n + \sum_{i=1}^{n-1} (c_2(n - i + 1) + (c_3 + c_4)(n - i)) =$$

Bubble Sort

BUBBLESORT(A, n)
for $i = 1$ to $n - 1$
  for $k = 1$ to $n - i$
    if $A[n - k + 1] < A[n - k]$
      exchange $A[n - k + 1]$ with $A[n - k]$

Say first line uses c_1 basic operations, second c_2, third c_3 and fourth c_4

Number of basic operations executed at most:

$$c_1 n + \sum_{i=1}^{n-1} (c_2(n - i + 1) + (c_3 + c_4)(n - i))$$

$$= c_1 n + \sum_{i=1}^{n-1} (c_2 + (c_2 + c_3 + c_4)(n - i))$$

$$= c_1 n + c_2(n - 1) + (c_2 + c_3 + c_4) \sum_{i=1}^{n-1} (n - i)$$

Bubble Sort

```
BUBBLESORT(A, n)
for i = 1 to n − 1
    for k = 1 to  n − i
        if A[n − k + 1]  < A[n − k]
            exchange A[n − k + 1] with A[n − k]
```

Say first line uses c_1 basic operations, second c_2, third c_3 and fourth c_4

Number of basic operations executed at most:

$$c_1 n + \sum_{i=1}^{n-1} (c_2(n-i+1) + (c_3 + c_4)(n-i))$$

$$= c_1 n + \sum_{i=1}^{n-1} (c_2 + (c_2 + c_3 + c_4)(n-i))$$

$$= c_1 n + c_2(n-1) + (c_2 + c_3 + c_4) \sum_{i=1}^{n-1} (n-i)$$

$$= c_1 n + c_2(n-1) + (c_2 + c_3 + c_4) \sum_{l=1}^{n-1} l \qquad \text{by change of variable setting } l = n - i$$

$$= c_1 n + c_2(n-1) + (c_2 + c_3 + c_4) \frac{(n-1)n}{2}$$

Bubble Sort

```
BUBBLESORT(A, n)
for i = 1 to n − 1
    for k = 1 to  n − i
        if A[n − k + 1]  < A[n − k]
            exchange A[n − k + 1] with A[n − k]
```

Say first line uses c_1 basic operations, second c_2, third c_3 and fourth c_4

Number of basic operations executed at most:

$$c_1 n + \sum_{i=1}^{n-1} (c_2(n-i+1) + (c_3+c_4)(n-i))$$

$$= c_1 n + \sum_{i=1}^{n-1} (c_2 + (c_2+c_3+c_4)(n-i))$$

$$= c_1 n + c_2(n-1) + (c_2+c_3+c_4) \sum_{i=1}^{n-1} (n-i)$$

$$= c_1 n + c_2(n-1) + (c_2+c_3+c_4) \sum_{k=1}^{n-1} l \qquad \text{by change of variable setting } l = n-i$$

$$= c_1 n + c_2(n-1) + (c_2+c_3+c_4)(n-1)n/2$$

Bubble Sort

```
BUBBLESORT(A, n)
for i = 1 to n − 1
    for k = 1 to  n − i
        if A[n − k + 1]  < A[n − k]
            exchange A[n − k + 1] with A[n − k]
```

Say first line uses c_1 basic operations (each time executed), second c_2, third c_3 and fourth c_4

Total number of basic operations executed at most:

$$c_1 n + \sum_{i=1}^{n-1} (c_2(n-i+1) + (c_3+c_4)(n-i))$$

$$= c_1 n + \sum_{i=1}^{n-1} (c_2 + (c_2+c_3+c_4)(n-i))$$

$$= c_1 n + c_2(n-1) + (c_2+c_3+c_4) \sum_{i=1}^{n-1} (n-i)$$

$$= c_1 n + c_2(n-1) + (c_2+c_3+c_4) \sum_{l=1}^{n-1} l \qquad \text{by change of variable setting } l = n-i$$

$$= c_1 n + c_2(n-1) + (c_2+c_3+c_4)(n-1)n/2$$

$$= (\frac{1}{2}c_2 + \frac{1}{2}c_3 + \frac{1}{2}c_4)n^2 + (c_1 + \frac{1}{2}c_2 - \frac{1}{2}c_3 - \frac{1}{2}c_4)n - c_2$$

Bubble Sort

```
BUBBLESORT(A, n)
for i = 1 to n − 1
    for k = 1 to  n − i
        if A[n − k + 1]  < A[n − k]
            exchange A[n − k + 1] with A[n − k]
```

Let's say first line uses c_1 basic operations, second c_2, third c_3 and fourth c_4

Total number of basic operations executed **at most**:

$$(\frac{1}{2}c_2 + \frac{1}{2}c_3 + \frac{1}{2}c_4)n^2 + (c_1 + \frac{1}{2}c_2 - \frac{1}{2}c_3 - \frac{1}{2}c_4)n - c_2$$

This is **equal** to number of basic operations executed when A has what property?

A is originally in decreasing order

Bubble Sort

BUBBLESORT(A, n)
for $i = 1$ to $n - 1$
  for $k = 1$ to $n - i$
    if $A[n - k + 1] < A[n - k]$
      exchange $A[n - k + 1]$ with $A[n - k]$

Say first line uses c_1 basic operations, second c_2, third c_3 and fourth c_4

Total number of basic operations executed **at most**:

$$(\frac{1}{2}c_2 + \frac{1}{2}c_3 + \frac{1}{2}c_4)n^2 + (c_1 + \frac{1}{2}c_2 - \frac{1}{2}c_3 - \frac{1}{2}c_4)n - c_2$$

This is **equal** to number of basic operations executed when A is in reverse sorted order

This quantity is $\theta(n^2)$

$\Rightarrow$The number of basic operations executed is $\theta(n^2)$, in the worst case

Bubble Sort

BUBBLESORT(A, n)
for $i = 1$ to $n - 1$
  for $k = 1$ to $n - i$
    if $A[n - k + 1] < A[n - k]$
      exchange $A[n - k + 1]$ with $A[n - k]$

Say first line uses c_1 basic operations, second c_2, third c_3 and fourth c_4

Total number of basic operations executed **at most**:

$$(\frac{1}{2}c_2 + \frac{1}{2}c_3 + \frac{1}{2}c_4)n^2 + (c_1 + \frac{1}{2}c_2 - \frac{1}{2}c_3 - \frac{1}{2}c_4)n - c_2$$

What about the best case?

Number of basic ops: $\Theta(n^2)$

Property of A: A is already sorted (inc order).

Bubble Sort Analysis revisited

Alternatively, could show worst-case run-time is $O(n^2)$ with a less exact calculation:

```
BUBBLESORT(A, n)
for i = 1 to n − 1                \\ do this at most n times
    for j = n downto  i + 1                \\ do this at most n times
        if A[j]  < A[j − 1]
            exchange A[j] with A[j − 1]
```

Bubble Sort Analysis revisited

Then, could show worst-case run-time is $\Omega(n^2)$ with a less exact calculation:

```
BUBBLESORT(A, n)
for i = 1 to n − 1                \\ for each i that is between 1 and n/2
    for j = n downto i+1                  \\ do this at least n/2 times
        if A[j] < A[j − 1]
            exchange A[j] with A[j − 1]
```

Since runtime is both $O(n^2)$ and $\Omega(n^2)$, it is $\theta(n^2)$

- Throughout this course, we will consider running "time" of an algorithm to be the number of basic operations it performs
- **Conclusion:**
  - Worst-case running time of this version of Bubblesort: $\theta(n^2)$
  - Best case running time of this version of Bubblesort: $\theta(n^2)$

Focus on worst-case

- From now on, if we say "The running time of Algorithm X is …" we mean the worst-case running time
- Why focus on worst case?
  - Might prefer to know about the "typical" running time, but usually not clear what that means.
  - Conservative

Review of asymptotic notation

- We will give running times as a function of input size, using asymptotic notation
- We use n to represent the input size, unless otherwise noted
- e.g.,
  - $\theta(n)$ linear
  - $\theta(n^2)$ quadratic
  - $\theta(2^n)$ exponential
  - $\theta(\log n)$ logarithmic

- We will give running times as a function of input size, using asymptotic notation
- We use n to represent the input size, unless otherwise noted
- e.g.
  - $\theta(n)$ linear
  - $\theta(n^2)$ quadratic
  - $\theta(2^n)$ exponential
  - $\theta(\log n)$ logarithmic

Big-Oh

- Big-Oh: $f(n) = O(g(n))$
  intuitively, $f(n)$ is "less than or equal to" $g(n)$
- Rule of thumb for getting big-Oh notation for sum of terms
  - remove lower order terms and multiplicative constants
  - e.g., $f(n) = 27n^2 + 5\log n + 39n + 4$

$O(n^2)$

  - why is this ok? We'll explain soon.

Big-Omega

- Big-Omega: $f(n) = \Omega(g(n))$
  intuitively, $f(n)$ is "greater than or equal to" $g(n)$
- Rule of thumb for getting big-Omega notation for expression that is sum of terms
  - remove lower order terms and multiplicative constants
  - e.g., $f(n) = 7n^2 + 5\log n + 39n + 4$

$\Omega(n^2)$

  - why is this ok? We'll explain soon.

Theta



- Theta (sometimes called Big-Theta)
  Intuitively, $f(n) = \theta(g(n))$ means they are asymptotically equal
- By definition, $f(n) = \theta(g(n))$ means
  - $f(n) = O(g(n))$
  - and $f(n) = \Omega(g(n))$
- Rule of thumb for getting Theta notation for expression that is sum of terms
  - Remove lower order terms and multiplicative constants
- e.g.
  - $f_1(n) = 38n^3 + 7n^2 + 1$
  - $f_2(n) = 10^{500}n^3 + 4n^2 + 24$

$f_1(n) = \Theta(n^3)$

$f_2(n) = \Theta(n^3)$

little-oh and little-omega

- little-oh: $f(n) = o(g(n))$

  Intuitively, $f(n)$ is much less than $g(n)$

  - e.g., if $f(n) = 57n^2 + 4n$ $f(n) = o(n^3)$
- little-omega: $f(n) = \omega(g(n))$

  Intuitively, $f(n)$ is much greater than $g(n)$

  - e.g., if $f(n) = n$ $f(n) = \omega(\sqrt{n})$
- little-oh and little-omega are opposites

  $f(n) = o(g(n))$ iff $g(n)$ is $\omega(f(n))$

Formal Definition of Big-Oh

f(n) and

- Note: We assume here that the functions $g(n)$ have positive values for all positive n
- Formally, $O(f(n))$ is a *class* (set) of functions
  - $O(g(n)) = \{f(n) : \exists c > 0, \exists n_0 > 0 \text{ such that } \forall n \geq n_0, \; f(n) \leq cg(n)\}$

Formal Definition of Big-Oh

- Note: We assume here that the functions $f(n)$ and $g(n)$ have positive values for all positive n (can also extend to functions $f(n)$ and $g(n)$ that are eventually positive)
- $O(g(n))$ is the following *class* (set) of functions
  - $O(g(n)) = \{f(n) : \exists c > 0, \exists n_0 > 0 \text{ such that } \forall n \geq n_0, \ \ 0 \leq f(n) \leq cg(n)\}$

    or less succinctly
  -

the set of functions f(n) such that there exist positive numbers c and n_0 satisfying the property that for all $n \geq n_0$, $0 \leq f(n) \leq c\,g(n)$

Formal Definition of Big-Oh

- $O(g(n)) = \{f(n) : \exists c > 0, \exists n_0 > 0 \text{ such that } \forall n \geq n_0,\ \ 0 \leq f(n) \leq cg(n)\}$
- We write $f(n) = O(g(n))$ to mean that $f(n)$ in this set
- Can we show formally that $3n^2 + 7n + 8 = O(n^2)$?
  Need to give appropriate values for c and n_0

$f(n)$ $\quad g(n)=n^2$

$3n^2 + 7n + 8 \leq 3n^2 + 7n^2 + 8n^2$

$= 18n^2 < 19n^2$ $\quad c = 19$

for all $n > 1$ $\quad n_0 = 1$

Formal Definition of Big-Oh

- $O(g(n)) = \{f(n) : \exists c > 0, \exists n_0 > 0 \text{ such that } \forall n \geq n_0, \ \ 0 \leq f(n) \leq cg(n)\}$
- We write $f(n) = O(g(n))$ to mean that $f(n)$ in this set
- Can we show formally that $n^2 + 7n + 5 = O(n)$?
  Need to give appropriate values for c and n_0

Formal Definition of Big-Omega

Recall: for $O(n)$ this was $f(n) \leq cg(n)$

- $\Omega(g(n)) = \{f(n) : \exists c > 0, \exists n_0 > 0 \text{ such that } \forall n \geq n_0, \; cg(n) \leq f(n)\}$



Formal Definition of little-oh

- $o(g(n)) = \{f(n) : \forall c > 0, \exists n_0 > 0$ such that $\forall n \geq n_0, \ 0 \leq f(n) < cg(n)\}$

Recall
$f(n) = O(g(n))$
Like $f(n)$ less than or equal to $g(n)$
$f(n) = o(g(n))$
like $f(n)$ is much less than $g(n)$

- Difference between this and definition of big-Oh?

$\forall c$ instead of $\exists c$

$3n^2 + 4 = O(n^2)$

$3n^2 + 4 \neq o(n^2)$

$\frac{1}{5} n^2 \neq o(n^2)$

- Under assumption that $f(n) > 0$ and g(n) > 0 when $n > 0$
  - $f(n) = o(g(n))$ **if**

$$lim_{n\to\infty}\frac{f(n)}{g(n)} = 0 \qquad \text{[limit test, limit needs to exist]}$$

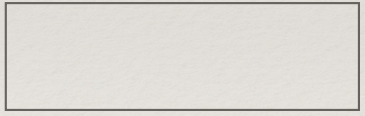
- e.g., $f(n) = 3n^2 + 4n + 189$ and $g(n) = n^3$

$$\lim_{n\to\infty} \frac{3n^2+4n+189}{n^3} = 0$$

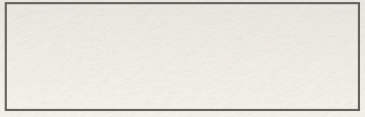
- Could define ω analogously to how we defined o
- Instead, just quote following fact:
  - Under assumption that $f(n) > 0$ and g(n) > 0 when $n > 0$
    - $f(n) = o(g(n))$ iff $g(n) = \omega(f(n))$
- Similar fact:
  - Under assumption that $f(n) > 0$ and g(n) > 0 when $n > 0$
    - $f(n) = O(g(n))$ iff $g(n) = \Omega(f(n))$

Polynomials of degree k

- Example: $f(n) = 3n^{25} + 2n^9 + 4$
- then $f(n) = o(n^b)$ for all $b > 25$



for all $b < 25$



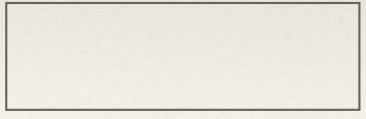
for $b = 25$

Polynomials of degree k

- Claim: Suppose $f(n)$ is a polynomial of degree k
  $f(n) = a_k n^k + a_{k-1} n^{k-1} + \ldots + a_1 n^1 + a_0$ where $a_k > 0$
  and $f(n) > 0$ for all $n > 0$
  - then $f(n) = o(n^b)$ for all $b > k$

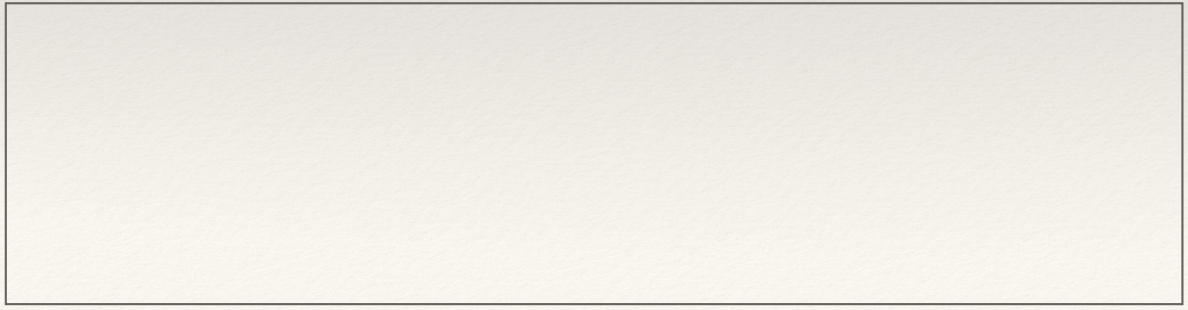
    for all $b < k$

    for $b = k$

Properties of the relation

- Big-Oh relation on functions
  - Reflexive
    - $f(n) = O(f(n))$
  - Transitive
    - if $f(n) = O(g(n))$ and $g(n) = O(h(n))$, then $f(n) = O(h(n))$
  - But is it symmetric?



Properties

- $f_1(n) = O(g(n))$ and $f_2(n) = O(g(n))$
  $\Rightarrow f_1(n) + f_2(n) = O(g(n))$
- $f_1(n) = O(g(n)), f_2(n) = O(g(n)), \ldots, f_{254}(n) = O(g(n))$
  $\Rightarrow$ 

Properties

- $f_1(n) = O(g(n))$ and $f_2(n) = O(g(n))$
  $\Rightarrow f_1(n) + f_2(n) = O(g(n))$
- More generally, for any constant $k > 0$,
  $f_1(n) = O(g(n))$, $f_2(n) = O(g(n))$, …, $f_k(n) = O(g(n))$
  $\Rightarrow$
- But, in general, won't be true if k is not a constant!